

untangle::executor

Generated by Doxygen 1.18.0

1 executor	1
2 Namespace Index	5
2.1 Namespace List	5
3 Class Index	7
3.1 Class List	7
4 File Index	9
4.1 File List	9
5 Namespace Documentation	11
5.1 untangle Namespace Reference	11
5.1.1 Detailed Description	11
6 Class Documentation	13
6.1 untangle::executor< taskT > Class Template Reference	13
6.1.1 Detailed Description	13
6.1.2 Constructor & Destructor Documentation	14
6.1.2.1 executor()	14
6.1.2.2 ~executor()	14
6.1.3 Member Function Documentation	15
6.1.3.1 add_task()	15
7 File Documentation	17
7.1 executor.hpp	17
Index	21

Chapter 1

executor

c++ thread pool executor based on async execution

The interface is one header that exposes a pool of workers, each one an **async** execution in continuous mode, fed by a shared queue of tasks.

This is the prototype, imported. It was written in `async/prototypes/` to answer whether execution is a usable building block for a pool — it is — and it is being hardened here. The behaviour below is what the tests state today, not a settled interface.

requirements

C++23, and the `async` submodule:

```
git submodule update --init --recursive
```

`async` carries its own actuator submodule, which is why the clone has to be recursive. Both are header-only; nothing is built or installed.

The headers report warnings with `std::println`, so they need a standard library that provides `<print>` — GCC 14 or newer.

example

```
#include <executor.hpp>
#include <functional>

int main() {
    untangle::executor<std::function<void(void)>> pool(4);

    for (int i = 0; i < 100; ++i) {
        pool.add_task([i] { work(i); });
    }

    return 0; // ~executor finishes what was submitted, then stops the workers
}
```

the task type

The pool is a template on the task it runs, the way an execution is a template on its action. One pool runs one signature, and the caller names it:

```
untangle::executor<std::function<int(void)>> pool(4); // returns are run and dropped
```

The task type does not have to be a `std::function`. What an execution requires of it is a nested `result_<_type`, which it reads rather than deduces, so any callable declaring one will do — and a pool built on such a callable never touches `std::function::result_type`, which the standard removed in C++20 and both `libc++` and `libstdc++` still supply by grace rather than contract.

```
struct my_task {
    using result_type = void;
    void operator()() const;
};

untangle::executor<my_task> pool(4);
```

A task type that takes arguments is refused, with a `static_assert` rather than an error inside `std::bind`: the pool queues tasks, and has nowhere to keep arguments for them until a worker frees up.

how work is placed

N executions run in continuous mode behind a shared `std::deque` of tasks.

Nothing overtakes. A task goes straight to a free worker only when the shared queue is empty; otherwise it joins the back of the queue. Finding a free worker does not let a task jump a queue that already has work in it.

Workers pull rather than being pushed to. When one reports its queue finished — on `_finished`, raised by execution the moment its list empties — it takes the front of the shared queue. That is what spreads the work, with no scheduling logic of its own: 400 tasks over 4 workers come out 100/100/100/100.

A worker is asked, not tracked. `execution::is_busy()` is what `add_task()` reads to find a free worker and what the destructor reads to decide the pool is idle. A tally kept here would be the pool's belief about its workers; this is the workers' own answer, and it cannot drift.

lifetime and failure

The destructor finishes what was submitted. It refuses further work, waits for the queue to empty and for every worker to go idle, and only then stops them. A task submitted before the pool goes out of scope has run by the time it does.

`add_task()` says whether the task was taken. It returns false once the pool is shutting down, which a task submitting more work from inside the pool can see.

A task that throws does not take the worker with it. `execution` catches it, reports it on `stderr`, and the worker carries on with the next task. Nothing reaches the submitter — a task that has something to report carries its own channel, because a continuous worker does not collect results.

building the tests

```
cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build
```

`test/CMakeLists.txt` fetches googletest at configure time, so the first configure needs network access.

sanitizers

EXECUTOR_SANITIZE builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DEXECUTOR_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are address, thread, undefined, or empty. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. address and thread instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

the reference

tools/make_doc.sh builds doc/refman.pdf from the header and this README. It needs doxygen, tectonic and python3, and it fails on any doxygen warning that is not an obsolete-tag notice, so an undocumented member or a broken \ref does not reach the reference.

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	A thread pool executor built out of <code>untangle::async::execution</code> objects	11
--------------------------	---	--------------------

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::executor< taskT >	
Runs tasks on a fixed pool of untangle::async::execution workers	13

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

executor.hpp	17
--	----

Chapter 5

Namespace Documentation

5.1 untangle Namespace Reference

A thread pool executor built out of `untangle::async::execution` objects.

Classes

- class `executor`
Runs tasks on a fixed pool of `untangle::async::execution` workers.

5.1.1 Detailed Description

A thread pool executor built out of `untangle::async::execution` objects.

Imported from the `async` repo's `prototypes/executor.hpp`, unchanged apart from its namespace and this comment. The hardening the prototype's README asks for has not been done yet.

The pool holds `N` executions in continuous mode and a shared queue of tasks. A task is handed straight to a free worker only when the shared queue is empty; otherwise it goes to the back of the queue, and a worker takes the front of the queue as it frees up. Nothing overtakes: finding a free worker does not let a task jump a queue that already has work in it.

Chapter 6

Class Documentation

6.1 `untangle::executor< taskT >` Class Template Reference

Runs tasks on a fixed pool of `untangle::async::execution` workers.

```
#include <executor.hpp>
```

Public Member Functions

- `executor` (`std::size_t worker_count`)
Starts `worker_count` executions in continuous mode.
- `~executor` ()
Finishes what has been submitted, then stops every worker.
- `bool add_task` (`taskT task`)
Queues a task, or hands it to a worker that has nothing to do.
- `std::size_t pending` () `const`
How many tasks are waiting for a worker.

6.1.1 Detailed Description

```
template<typename taskT>  
class untangle::executor< taskT >
```

Runs tasks on a fixed pool of `untangle::async::execution` workers.

Template Parameters

<code>taskT</code>	The type of the task the pool runs, specified as <code>std::function<...></code> like the execution it is handed to. One pool runs one signature, because an execution's action type is fixed by its own template argument.
--------------------	---

Remarks

A task that needs to return something carries its own channel: a continuous worker does not collect results, so a non-void return is run and dropped.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 executor()

```
template<typename taskT>
untangle::executor< taskT >::executor (
    std::size_t worker_count)    [inline], [explicit]
```

Starts worker_count executions in continuous mode.

Parameters

worker_count	- How many workers to run. Must not be zero.
--------------	--

Exceptions

std::invalid_argument	- worker_count is zero. Such a pool would take work and never run it, so it is refused before any worker starts rather than left to hang in the destructor.
-----------------------	---

Remarks

std::thread::hardware_concurrency() returns 0 when it cannot tell how many cores there are, and a caller passing that straight through is the likely way in. Choosing a default then is the caller's call, not the pool's.

6.1.2.2 ~executor()

```
template<typename taskT>
untangle::executor< taskT >::~~executor ()    [inline]
```

Finishes what has been submitted, then stops every worker.

Both waits are unbounded - a pool that abandoned a running task would be worse than one that waits for it - and both report on stderr rather than stalling in silence, naming the workers they are still waiting on.

Remarks

Every worker has left its thread before the first execution is destroyed. ~execution() waits only for its own worker, so destroying them one at a time would free the first while another's thread was still reading it through nothing_running(); the poll waits for all of them. It is this pool's own poll, so a second pool running alongside it is not waited for.

6.1.3 Member Function Documentation

6.1.3.1 add_task()

```
template<typename taskT>  
bool untangle::executor< taskT >::add_task (  
    taskT task)    [inline]
```

Queues a task, or hands it to a worker that has nothing to do.

Returns

true - the task was accepted. false - the pool is shutting down and refused it.

The documentation for this class was generated from the following file:

- executor.hpp

File Documentation

```
00001 // Copyright (c) 2026 Nicolae Popescu. MIT License.
00002
00014 #pragma once
00015
00016 #include <algorithm>
00017 #include <async.hpp>
00018 #include <chrono>
00019 #include <condition_variable>
00020 #include <cstdint>
00021 #include <deque>
00022 #include <functional>
00023 #include <memory>
00024 #include <mutex>
00025 #include <print>
00026 #include <stdexcept>
00027 #include <string>
00028 #include <thread>
00029 #include <type_traits>
00030 #include <utility>
00031 #include <vector>
00032
00033 namespace untangle {
00034
00045 template <typename taskT>
00046 class executor {
00047     // add_action() is the only way in, and the pool calls it with no arguments to bind: what it
00048     // queues is the task itself, and there is nowhere to keep arguments alongside it until a worker
00049     // is free. Stated here so a pool on std::function<void(int)> says why rather than failing inside
00050     // std::bind.
00051     static_assert(std::is_invocable_v<taskT>,
00052         "executor: the task type must be callable with no arguments");
00053
00054 public:
00067 explicit executor(std::size_t worker_count) {
00068     if (worker_count == 0) {
00069         throw std::invalid_argument("executor: worker_count must not be zero");
00070     }
00071
00072     workers_.reserve(worker_count);
00073
00074     for (std::size_t index = 0; index < worker_count; ++index) {
00075         auto next = worker_execution::create_instance("pool_worker_" + std::to_string(index));
00076
00077         // How a worker learns there is more waiting for it. is_busy() answers whether a worker has
00078         // room; this is what tells the pool the moment one frees up, without anybody polling.
00079         next->on_finished = [this, index] { take_next_task(index); };
00080
00081         // A worker runs in a detached thread and cannot be joined, so asking whether it is still
00082         // running is the only way to wait for one, and the poll answers that for any number of them
00083         // at once. A poll answers for the executions added to it, and this one is added the pool's
00084         // own workers and nothing else, so what the destructor waits for is exactly these. Nothing
00085         // here has to unregister - the record runs both ways, so whichever of the two dies first
00086         // takes itself out of the other.
00087         poll_.add(*next);
00088
00089         workers_.push_back(std::move(next));
00090         workers_.back()->start();
    }
```

```

00091     }
00092 }
00093
00106 ~executor() {
00107     {
00108         std::unique_lock<std::mutex> lock(mutex_);
00109         accepting_ = false;
00110
00111         // Unbounded, and reported rather than bounded. Both waits below keep waiting for as long as
00112         // it takes; what they stop doing is keeping it to themselves.
00113         auto interval = std::chrono::milliseconds(report_first_ms);
00114         auto waited = std::chrono::milliseconds(0);
00115
00116         while (!finished_cv_.wait_for(lock, interval,
00117             [this] { return pending_.empty() && nothing_running(); })) {
00118             waited += interval;
00119
00120             // The lock is held here, which is what makes both reads consistent with the predicate that
00121             // just failed. is_busy() takes each execution's action_mutex_ under this one, which is the
00122             // same order give_to_worker() uses - no new lock order is introduced by reporting.
00123             std::println(
00124                 stderr, "executor: still waiting to finish after {}s - {} queued, {} in a task{}",
00125                 waited.count() / 1000, pending_.size(), count_workers(&worker_execution::is_busy),
00126                 name_workers(&worker_execution::is_busy));
00127
00128             interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00129         }
00130     }
00131
00132     // Outside the lock: stop() wakes each worker, and a worker waking up runs on_finished, which
00133     // wants this mutex.
00134     for (auto& one : workers_) {
00135         one->stop();
00136     }
00137
00138     // The poll stands in for a join. Until it reports idle, a worker can still be inside
00139     // take_next_task(), reading every execution through nothing_running() and is_busy() - which is
00140     // what makes destroying them here a use-after-free rather than a teardown.
00141     //
00142     // A different question from the finish above, and reported as one: that wait is about workers
00143     // still inside a task, this is about workers whose thread has not left. A worker can be idle
00144     // and still be here.
00145     auto interval = std::chrono::milliseconds(report_first_ms);
00146     auto waited = std::chrono::milliseconds(0);
00147
00148     while (poll_.is_running()) { // time of check
00149         const auto tick = std::chrono::milliseconds(poll_interval_ms);
00150         std::this_thread::sleep_for(tick);
00151         waited += tick;
00152
00153         if (waited >= interval) {
00154             std::println(stderr, "executor: still waiting to stop after {}s - {} not left its thread{}",
00155                 waited.count() / 1000, count_workers(&worker_execution::is_running),
00156                 name_workers(&worker_execution::is_running));
00157
00158             interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00159         }
00160     }
00161
00162     // time of use: no worker thread is left to reach workers_, so clearing it races nothing.
00163     workers_.clear();
00164 }
00165
00171 bool add_task(taskT task) {
00172     std::lock_guard<std::mutex> lock(mutex_);
00173
00174     if (!accepting_) {
00175         return false;
00176     }
00177
00178     // The queue comes first whenever it has anything in it: a task that arrives while others are
00179     // waiting does not get to overtake them just because a worker happens to be free.
00180     if (pending_.empty()) {
00181         for (std::size_t index = 0; index < workers_.size(); ++index) {
00182             if (!workers_[index]->is_busy()) {
00183                 give_to_worker(index, std::move(task));
00184                 return true;
00185             }
00186         }
00187     }
00188
00189     pending_.push_back(std::move(task));
00190     return true;
00191 }
00192
00196 std::size_t pending() const {
00197     std::lock_guard<std::mutex> lock(mutex_);

```

```

00198     return pending_.size();
00199 }
00200
00201 private:
00202     using worker_execution = untangle::async::execution<taskT>;
00203
00210 void give_to_worker(std::size_t index, taskT task) {
00211     // add_action() takes the execution's own action_mutex while this holds mutex_. That is only
00212     // safe in one direction, and it holds: a worker calls on_finished with action_mutex released,
00213     // so it never takes mutex_ while holding action_mutex, and the two never form a cycle.
00214     workers_[index]->add_action(std::move(task));
00215 }
00216
00220 void take_next_task(std::size_t index) {
00221     std::lock_guard<std::mutex> lock(mutex_);
00222
00223     if (!pending_.empty()) {
00224         taskT next = std::move(pending_.front());
00225         pending_.pop_front();
00226         give_to_worker(index, std::move(next));
00227         return;
00228     }
00229
00230     if (nothing_running()) {
00231         finished_cv_.notify_all();
00232     }
00233 }
00234
00241 bool nothing_running() const {
00242     for (const auto& one : workers_) {
00243         if (one->is_busy()) {
00244             return false;
00245         }
00246     }
00247     return true;
00248 }
00249
00250 std::size_t count_workers(bool (worker_execution::*state)() const) const {
00251     std::size_t count = 0;
00252
00253     for (const auto& one : workers_) {
00254         if ((one.get()->*state)()) {
00255             ++count;
00256         }
00257     }
00258     return count;
00259 }
00260
00267 std::string name_workers(bool (worker_execution::*state)() const) const {
00268     std::string names;
00269
00270     for (const auto& one : workers_) {
00271         if ((one.get()->*state)()) {
00272             names += names.empty() ? " : " : ", ";
00273             names += one->name;
00274         }
00275     }
00276     return names;
00277 }
00278
00289 static constexpr int poll_interval_ms = 50;
00290
00297 static constexpr int report_first_ms = 1000;
00298 static constexpr int report_max_ms = 30000;
00299
00300 mutable std::mutex mutex_;
00301 std::condition_variable finished_cv_;
00302
00313 async::execution_poll poll_;
00314
00315 std::deque<taskT> pending_;
00316 std::vector<std::shared_ptr<worker_execution> > workers_;
00317 bool accepting_ = true;
00318 };
00319
00320 } // namespace untangle

```


Index

- `~executor`
 - `untangle::executor< taskT >`, [14](#)
- `add_task`
 - `untangle::executor< taskT >`, [15](#)
- `executor`, [1](#)
 - `untangle::executor< taskT >`, [14](#)
- `untangle`, [11](#)
- `untangle::executor< taskT >`, [13](#)
 - `~executor`, [14](#)
 - `add_task`, [15](#)
 - `executor`, [14](#)